ARGUS: Evidence Context Construction and Adversarial Validation for Vulnerability Detection

Anonymous Author(s)

Anonymous Institution

Abstract. Software vulnerability detection with Large Language Models (LLMs) remains challenging because reliable predictions require both relevant program context and reasoning grounded in concrete program evidence. Existing approaches use program slicing or repository context to reduce irrelevant code, yet the resulting context may still miss security-critical dependencies, and potential safeguards are not always explicitly validated before classification. We present **ARGUS**, a program-grounded framework that combines Code Property Graph (CPG) analysis with opposing-evidence validation. ARGUS identifies security-sensitive operations and reconstructs candidate-specific contexts containing the data, control, and interprocedural information relevant to each candidate. Rather than directly classifying the reconstructed context, ARGUS separately examines evidence supporting the vulnerability condition and counter-evidence that may invalidate it, and reconciles both through an adjudication step. On the evaluated PrimeVul artifact, ARGUS achieves 100.00% Recall, 57.58% Precision, and 73.08% F1-score. On SVEN, it achieves 75.27% Recall, 52.63% Precision, and 61.95% F1-score. These results characterize ARGUS as a recall-oriented detector across both evaluation settings, while also highlighting false-positive reduction as an important direction for further improvement.

Keywords: Software Vulnerability Detection · Large Language Models · Code Property Graphs · Multi-Agent Systems

1 Introduction

Software vulnerabilities remain a persistent threat to modern software systems and can lead to serious security and operational consequences [10]. Detecting them before deployment is therefore a long-standing objective of software security research. Despite substantial progress, reliably identifying vulnerable source code remains difficult, particularly when the vulnerability depends on non-local data flow, path conditions, or project-specific semantics [9, 7].

Learning-based vulnerability detection was introduced partly to move beyond rigid rule matching. Early work modeled source code as token sequences, while later approaches incorporated graph structure and program dependencies [7, 14, 9]. More recently, LLMs have been applied through prompting, in-context learning, and task-specific fine-tuning, offering stronger semantic reasoning over

code [10, 1]. A parallel line of work combines LLMs with static analysis so that security-relevant program information is made explicit before the model is asked to reason about a vulnerability [8].

A central difficulty in this setting is the construction of the reasoning context. Supplying the complete function or repository introduces large amounts of irrelevant code, whereas aggressive reduction may omit the relations that determine whether a suspicious operation is actually unsafe. Program slicing helps reduce irrelevant context [5], but a smaller slice is not necessarily a sufficient security context. In addition, LLMs can still rely on superficial cues or unsupported assumptions when the evidence needed to confirm or refute a vulnerability is not made explicit [10]. Reliable detection therefore requires both a focused program context and a validation process that remains tied to that context.

We address this problem with **ARGUS**, a three-stage framework for sensitive operation identification, program context construction, and vulnerability validation. ARGUS first maps concrete CPG invocation nodes to security-relevant operations using a curated knowledge base, with an LLM-assisted fallback when exact API matching yields no candidate. For each candidate, ARGUS reconstructs the surrounding data, control, and call relations, supplements the slice with source-level and bounded interprocedural evidence, and produces a compact context for validation. Finally, a Prosecutor and a Defender analyze the same context for supporting and counter-evidence, and an Adjudicator determines whether the vulnerability condition remains feasible.

The main contributions of this work are as follows:

Security-Sensitive Candidate Identification. We ground candidate discovery in concrete CPG call sites and combine exact API matching with a constrained LLM fallback for project-specific operations.

Program Context Construction. We construct candidate-specific contexts from backward and forward slices, control dependencies, source-level evidence, and bounded interprocedural relations rather than presenting the LLM with raw source code.

Adversarial Vulnerability Validation. We validate each candidate through opposing evidence analyses over the same program context and reconcile them with a separate Adjudicator to produce the final verdict.

2 Related Work

2.1 LLM-Based Vulnerability Detection

Recent studies apply LLMs to vulnerability detection using zero-shot and few-shot prompting, Chain-of-Thought reasoning, in-context learning, task-specific fine-tuning, and retrieval-augmented generation [10, 1, 2]. These methods improve semantic reasoning and make it possible to incorporate security knowledge without relying entirely on manually written rules. Vul-RAG, for example, retrieves

vulnerability causes, triggering conditions, and historical fixes to support inference [2]. However, when the model is given raw or weakly structured code, it must still infer the relevant operation, dependencies, triggering conditions, and safeguards at the same time, making the final decision sensitive to irrelevant context and unsupported assumptions.

2.2 Program-Grounded Vulnerability Analysis

Recent systems increasingly combine LLM reasoning with static program analysis. IRIS uses LLMs to infer security specifications that are checked through repository-scale CodeQL analysis [8]. LLMxCPG constructs vulnerability-focused CPG slices that retain relevant control and data dependencies [5], while RepoAudit performs demand-driven repository exploration using interprocedural data-flow and path evidence [3]. Because LLMxCPG is the closest prior approach to ARGUS in its use of CPG-guided context construction, we select it as the primary baseline and reproduce the released implementation on the same evaluation artifacts used by ARGUS; the reproduction protocol is described in Section 4.1.

2.3 Multi-Agent Vulnerability Detection

Multi-agent systems decompose vulnerability analysis across specialized roles. VulTrial uses a mock-court process with opposing agents and a review board [12]; MulVul employs vulnerability-specific agents and retrieval-augmented reasoning [13]; MAVUL uses contextual reasoning and iterative refinement [6]; and VulAgent separates vulnerability discovery from hypothesis construction and validation against project context [11]. ARGUS differs primarily in the evidence supplied to its agents: the Prosecutor and Defender receive the same statically reconstructed program context and are asked to test competing interpretations of that evidence.

Table 1 positions ARGUS between two complementary directions. LLMxCPG emphasizes program-analysis-driven context construction through CPG-based slicing, whereas VulTrial emphasizes structured multi-agent reasoning. ARGUS combines these ideas by first reconstructing candidate-specific program context with static analysis and then validating the candidate through supporting- and counter-evidence reasoning over the same context.

3 Approach

3.1 Overview

A sensitive operation indicates where a vulnerability may manifest, but it is rarely sufficient to determine whether the program is actually unsafe. The decision usually depends on the values reaching the operation, the conditions under which it executes, and, in some cases, behavior outside the target function. At the same time, providing the complete program to an LLM introduces substantial

Table 1. Positioning of ARGUS relative to LLMxCPG and VulTrial.

Aspect	**LLMxCPG**	**VulTrial**	**ARGUS**
Program analysis	CPG-guided static analysis	No dedicated static-analysis stage	CPG-guided static analysis
Context construction	Vulnerability-focused program slices	Source code provided to role-specific agents	Candidate-specific program context
Reasoning organization	Downstream vulnerability classification	Multi-agent mock-court interaction	Prosecutor–Defender–Adjudicator validation
Evidence grounding	Reasoning over CPG-derived slices	Reasoning over source code and inter-agent arguments	Reasoning over a shared reconstructed program context
Counter-evidence handling	Considered within downstream reasoning	Introduced through opposing agent roles	Explicitly evaluated against supporting evidence
Decision mechanism	Vulnerability classification	Review-board decision	Evidence adjudication

irrelevant context. ARGUS addresses this tension by separating localization, context construction, and validation.

Given a target program P, ARGUS constructs its Code Property Graph $G = \mathrm{CPG}(P)$. For each sensitive operation a_i, the system associates a vulnerability type t_i and forms a candidate

$$h_i = (a_i, t_i). \tag{1}$$

Stage 2 builds a candidate-specific context C_i, and Stage 3 produces

$$V_i \in \{\textsc{Vulnerable}, \textsc{Benign}\}. \tag{2}$$

Figure 1 summarizes the workflow.

3.2 Sensitive Operation Identification

Stage 1 identifies program locations at which security-relevant behavior may occur. These locations are treated as *sensitive operations*; their presence does not by itself imply a vulnerability.

Security Knowledge Base ARGUS uses a curated security knowledge base to associate concrete security-sensitive operations with vulnerability categories. The knowledge base is fixed before the final evaluation and is not updated from model predictions or test-set errors. It separates exact API mappings

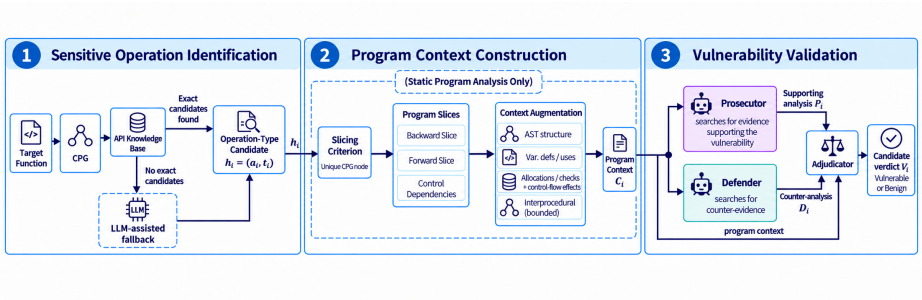


Fig. 1. Overview of the ARGUS framework.

Table 2. Schema of the ARGUS security knowledge base.

Field	Description
Knowledge ID	Unique identifier of a knowledge entry.
CWE	Associated CWE category.
Weakness	Human-readable vulnerability type.
Operation kind	Semantic class of the sensitive operation.
API / pattern	Concrete API names or operation patterns associated with the entry.
Operand roles	Security-relevant operands, such as buffer, index, size, or object pointer.
Selection rule	Condition under which a concrete CPG call is treated as a candidate.
Exclusion rule	Conditions under which a matched operation should not be selected.
Evidence to inspect	Program properties relevant to downstream vulnerability reasoning.

from semantic reasoning cards: exact mappings support deterministic matching between normalized CPG call names and CWE categories, while semantic cards describe the operation kind, operand roles, candidate-selection rule, exclusion rule, and program evidence relevant to reasoning when no exact API match is available.

For example, an indexed-read entry for CWE-125 identifies the concrete memory-read operation as the candidate, records the base object and index as operand roles, excludes index calculation or validation statements from candidate selection, and directs later reasoning toward the index value, object extent, backward data flow, and relevant bounds checks. The knowledge base is constructed from public CWE descriptions, documented API semantics, and security-relevant operation patterns rather than from the final benchmark predictions. Any entries refined during development are frozen before the reported evaluation.

Candidate Identification ARGUS extracts non-operator invocation nodes from the CPG and normalizes their callee names. Each invocation is first matched

against the exact API mappings in the knowledge base; a successful match creates one candidate for each associated CWE. When no exact candidate is found, ARGUS invokes a constrained LLM fallback that receives the target function, the concrete invocation nodes extracted from the CPG, and the semantic reasoning cards from the knowledge base. The model may select at most three operation–CWE hypotheses, but it cannot introduce arbitrary source expressions: each selected operation must correspond to an existing CPG node and is revalidated before Stage 2. Stage 1 therefore performs localization only; vulnerability classification is deferred until the surrounding program behavior has been reconstructed.

3.3 Program Context Construction

The statement containing a sensitive operation rarely contains all of the evidence required for a security decision. A memory copy, for example, may depend on the origin of its length argument, the destination allocation, preceding bounds checks, and the effect of those checks on control flow. Similar non-local relationships arise in integer-overflow and object-lifetime vulnerabilities. ARGUS therefore builds a candidate-specific context instead of analyzing either the isolated call or the complete function.

Program Slicing For each candidate $h_i = (a_i, t_i)$, ARGUS resolves a_i to a unique CPG node and uses it as the slicing criterion. Candidates that cannot be mapped unambiguously are not forwarded to later stages. Backward slicing recovers definitions and computations that may influence the candidate, forward slicing captures operations affected by it, and control dependencies preserve conditions that determine whether relevant statements can execute. The dependency context is

$$C_i^{dep} = C_i^{back} \cup C_i^{forward} \cup C_i^{ctrl}, \tag{3}$$

where C_i^{back}, $C_i^{forward}$, and C_i^{ctrl} denote the backward slice, forward slice, and relevant control dependencies. ARGUS also tracks variables involved in the candidate to recover their definitions and uses, including size computations, indexes, offsets, and related allocations.

Context Augmentation Dependency traversal alone may miss information that is semantically relevant to the security decision. ARGUS therefore augments the slice with complementary source-level and interprocedural evidence. At the source level, the AST is used to recover declarations, definitions, allocations, operand uses, and conditional checks. For each check, ARGUS retains both the predicate and its control-flow consequence because the existence of a condition does not imply that it actually blocks the unsafe path.

When relevant information crosses a function boundary, ARGUS performs bounded interprocedural analysis. Caller–callee relations and argument–parameter

mappings are followed on demand when they help interpret the candidate. This can recover the origin of an argument in a caller, the semantics of a project-specific wrapper, or a callee effect on an object used by the target function. Repository code is not added indiscriminately; only information connected to the candidate through relevant program relations is retained.

The extracted statements are deduplicated, multi-line statements are reconstructed, and the remaining code is ordered by source location. If the context exceeds the configured budget, ARGUS prioritizes the candidate, its operands, relevant checks, and interprocedural dependencies. Context construction is performed entirely through program analysis; no LLM is used in Stage 2.

3.4 Vulnerability Validation

The context produced by Stage 2 can contain evidence both for and against a vulnerability. A suspicious value may reach a sensitive operation while an earlier condition constrains the value or terminates the corresponding path. ARGUS therefore separates the two lines of reasoning.

The *Prosecutor* examines C_i for evidence that supports the vulnerability condition, such as unsafe value propagation, insufficient bounds, arithmetic overflow, invalid object lifetime, or the absence of an effective check. The *Defender* examines the same context for counter-evidence, including bounds checks, input validation, path constraints, terminating error handling, object reinitialization, ownership transfer, and safe cleanup. Both roles operate on exactly the same program context, so their conclusions differ in analytical objective rather than in available information.

Their analyses are then passed, together with C_i, to an *Adjudicator*. The Adjudicator checks whether the claims from either side are supported by the reconstructed program context and determines whether the vulnerability condition remains feasible after the relevant checks and constraints are considered:

$$V_i = \text{Judge}(P_i, D_i, C_i), \qquad V_i \in \{\textsc{Vulnerable}, \textsc{Benign}\}, \tag{4}$$

where P_i and D_i denote the Prosecutor and Defender analyses. If a function contains multiple candidates, ARGUS validates them independently and classifies the function as Vulnerable when at least one candidate is vulnerable; otherwise it returns Benign.

4 Evaluation

We evaluate ARGUS through three research questions:
RQ1: Vulnerability Detection Performance. How effective is ARGUS in detecting vulnerable functions?
RQ2: Performance across CWE Categories. How does ARGUS perform across different vulnerability categories?
RQ3: Error Analysis. What are the main sources of incorrect predictions made by ARGUS?

Table 3. Composition of the released evaluation artifacts used in our reproduced experiments.

Dataset	CWE	Vulnerable	Benign	Total
PrimeVul	CWE-125	10	10	20
	CWE-190	10	10	20
	CWE-415	10	10	20
	CWE-416	10	10	20
	Total	40	40	80
SVEN	CWE-125	50	41	91
	CWE-190	35	35	70
	CWE-416	44	49	93
	Total	129	125	254

4.1 Experimental Setup

Datasets and Evaluation-Set Construction

PrimeVul. PrimeVul is a large-scale real-world C/C++ vulnerability dataset designed to reduce label noise, code duplication, and temporal leakage [1]. The original dataset contains 6,968 vulnerable functions collected from 6,827 vulnerability-fixing commits and spans 140 CWE categories. In our experiments, we use the PrimeVul evaluation artifact released with LLMxCPG [5]. This evaluation set contains 80 labeled instances, including 40 vulnerable and 40 benign samples. It covers CWE-125, CWE-190, CWE-415, and CWE-416, with 10 vulnerable and 10 benign instances for each category. The same evaluation set is used for ARGUS and our LLMxCPG experiments to ensure a consistent comparison.

SVEN. SVEN is a manually curated function-level vulnerability dataset constructed from real-world security-fixing commits [4]. For our experiments, we use the SVEN evaluation artifact released with LLMxCPG [5]. The evaluation set contains 254 labeled instances across CWE-125, CWE-190, and CWE-416. Specifically, it contains 50 vulnerable and 41 benign instances for CWE-125, 35 vulnerable and 35 benign instances for CWE-190, and 44 vulnerable and 49 benign instances for CWE-416. Each function is treated as an independent classification input. Vulnerable–fixed pair information, patch differences, and pair correspondence are not provided to ARGUS during inference.

Duplicate and Leakage Control. The sample list is frozen before inference and reused across directly compared methods. We audit exact and normalized-code duplicates in the released artifacts and do not use vulnerable–fixed relations, patch information, CVE descriptions, or gold CWE labels for inference or candidate selection. When development examples or few-shot demonstrations are used by a reproduced method, samples sharing the same normalized-code identity must be excluded from its evaluation partition. This separation is necessary because the released artifacts and the aggregate dataset statistics in the original LLMxCPG paper are not fully aligned.

ARGUS Configuration ARGUS is implemented in Python and uses Joern to construct and query CPGs for C/C++ programs. All LLM-based components use **Qwen3-32B**; no task-specific fine-tuning is performed on PrimeVul or SVEN. Temperature is set to 0 whenever supported, and the same prompts and model configuration are used across the two released evaluation artifacts. Candidate-specific contexts are constructed with backward slicing, forward slicing, control dependencies, source-level augmentation, and bounded interprocedural analysis. The pipeline stores candidates, reconstructed contexts, intermediate agent outputs, and final predictions so that each classification can be traced through the three stages.

Baselines and Reproduction Protocol LLMxCPG [5] is the primary baseline because it is the closest prior approach in combining CPG-based program analysis with LLM-based vulnerability detection. We reproduce the authors' released implementation and use the released evaluation artifacts rather than assuming that the repository data exactly matches the aggregate dataset distribution reported in the paper. For controlled comparisons, ARGUS and the reproduced LLMxCPG pipeline are executed on the same frozen sample identities and evaluated with the same gold labels and metric implementation. No sample is added or removed based on model outputs. Execution failures are recorded explicitly rather than being silently discarded.

The reproduction follows the released LLMxCPG workflow and preserves the original prompt templates and model-specific processing where available. The original paper reports QwQ-32B-Preview as the base model for LLMxCPG-D, with LoRA rank 8, LoRA alpha 4, and a learning rate of 10^{-4} during fine-tuning [5]. For our final artifact, we additionally record the exact repository revision, released checkpoint, Joern version, decoding configuration, maximum input and output lengths, context budget, and evaluation script used in the local reproduction. These implementation identifiers are reported with the released code artifact to distinguish our reproduced results from values quoted from the original publication.

Result Sources. For the primary baseline, we distinguish values obtained from our local LLMxCPG reruns from values quoted from prior publications. Direct comparisons with ARGUS use the same released evaluation artifact whenever a local baseline rerun is available. Values taken from an original publication are retained only as external reference points and are not treated as controlled same-sample comparisons. This distinction is necessary because the dataset composition reported in the LLMxCPG paper does not fully match the released evaluation artifacts used in our experiments.

Performance Metrics and Failure Handling We report Accuracy (ACC), Precision (P), Recall (R), and F1-score (F1) for function-level binary classification. Accuracy captures overall correctness; Precision reflects the reliability of vulnerable predictions; Recall measures the proportion of vulnerable functions

Table 4. Configuration recorded for the reproduced LLMxCPG baseline.

Item	Reproduction setting
Source code	Authors' released LLMxCPG implementation
Repository revision	Recorded from the exact snapshot used in our experiments
Base checkpoint	QwQ-32B-Preview for LLMxCPG-D
Fine-tuning	LoRA rank 8, alpha 4, learning rate 10^{-4} as reported by LLMxCPG
Prompt templates	Released LLMxCPG prompts, unchanged where available
Joern / CPG pipeline	Same released workflow used by the reproduced baseline
Decoding parameters	Recorded from the local inference configuration
Input/output limits	Recorded from the local inference configuration
Context budget	Recorded from the released/local reproduction configuration
Evaluation samples	Same frozen released sample identities used by ARGUS

Table 5. Results on the released PrimeVul evaluation artifact. ARGUS metrics are computed over 77 valid predictions from 80 submitted inputs.

Method	Result source	ACC (%)	P (%)	R (%)	F1 (%)
LLMxCPG	Original paper / Local rerun	72.50	**100.00**	45.00	62.06
ARGUS	Local run on released artifact	63.64	57.58	**100.00**	**73.08**

detected; and F1 summarizes the balance between Precision and Recall. For every table, the metric denominator is the number of samples that produced an evaluable binary prediction. Pipeline failures are reported separately instead of being silently removed. In the current PrimeVul ARGUS run, 80 inputs were submitted but 77 produced valid final predictions; the reported 63.64% Accuracy, 57.58% Precision, 100.00% Recall, and 73.08% F1-score are therefore computed over those 77 valid predictions, corresponding to TP=38, FP=28, TN=11, and FN=0. This distinction is stated explicitly to avoid interpreting these metrics as an 80-sample confusion matrix.

4.2 Vulnerability Detection Performance (RQ1)

RQ1 examines the overall function-level detection performance of ARGUS. Because the available baseline settings differ across PrimeVul and SVEN, the two datasets are reported separately.

PrimeVul. The LLMxCPG values in Table 5 are the results reported in the original publication and are retained as a published reference because the released PrimeVul artifact differs from the aggregate distribution described in the paper. ARGUS produces 77 valid predictions from the 80 submitted instances, with TP=38, FP=28, TN=11, and FN=0; its metrics are computed over these 77 predictions. A controlled same-artifact comparison requires the locally rerun LLMxCPG result on the released artifact.

Table 6. Function-level vulnerability detection performance on the released SVEN evaluation artifact.

Method	ACC (%)	P (%)	R (%)	F1 (%)
VulSim	33.00	31.00	31.00	31.00
VulBERTA-CNN	50.00	51.00	38.00	44.00
VulBERTA-MLP	50.00	50.00	37.00	43.00
ReGVD	51.00	53.00	46.00	55.00
LLMxCPG	**62.61**	**66.67**	42.48	51.89
ARGUS	55.21	52.63	**75.27**	**61.95**

Table 7. Performance Metrics of ARGUS by CWE Category on PrimeVul.

CWE	ACC (%)	P (%)	R (%)	F1 (%)
CWE-125	65.00	58.82	100.00	74.07
CWE-190	57.89	55.56	100.00	71.43
CWE-415	57.89	52.94	100.00	69.23
CWE-416	73.68	64.29	100.00	78.26

SVEN. The VulSim, VulBERTA-CNN, VulBERTA-MLP, and ReGVD values are taken from the original LLMxCPG evaluation, whereas LLMxCPG is rerun locally on the released SVEN artifact used by ARGUS. The direct same-artifact comparison is therefore between LLMxCPG and ARGUS. ARGUS achieves 75.27% Recall and 61.95% F1-score, while LLMxCPG attains higher Accuracy and Precision.

4.3 Performance across CWE Categories (RQ2)

RQ2 examines whether ARGUS behaves differently across vulnerability categories that require different forms of program reasoning. We group the predictions from RQ1 by CWE and report the same classification metrics.

Across the four PrimeVul categories, the current breakdown preserves the aggregate behavior observed in RQ1: Recall remains high, while Precision varies by category. CWE-125 requires reasoning about memory extents, indexes, offsets, and access bounds; CWE-190 depends on arithmetic value propagation and constraints; whereas CWE-415 and CWE-416 require reasoning about object lifetime, deallocation, aliases, and subsequent use.

4.4 Error Analysis (RQ3)

We analyze misclassified predictions at the three stages of ARGUS and organize the observed failure patterns accordingly.

Sensitive Operation Identification. Stage 1 can select an operation that is security-sensitive and plausible for the target CWE but is not the operation that determines the actual vulnerability condition. Because the selected operation becomes the slicing criterion, this can redirect later analysis toward dependencies that are

valid but not decisive. Project-specific wrappers are particularly difficult when their security semantics are not evident from the call site.

Program Context Construction. A relevant candidate can still lead to an incorrect decision if Stage 2 omits a dependency, alias relation, interprocedural constraint, or effective security check. One recurring issue is recovering a guard without the control-flow consequence that makes the guard effective. In that case, the context contains the condition but not the evidence that it blocks the unsafe path.

Vulnerability Validation. Errors also occur when the necessary information is present but Stage 3 weighs it incorrectly. This is consistent with the precision–recall profile in RQ1: ARGUS often recognizes potentially unsafe behavior, yet effective bounds checks, terminating paths, or safe lifetime transitions may receive insufficient weight during adjudication.

Overall, the remaining errors arise from both evidence recovery and evidence interpretation. Reducing false positives therefore requires not only broader context coverage but also more reliable recognition of when recovered counter-evidence actually invalidates the suspected vulnerability condition.

5 Discussion

5.1 Limitations

Sensitive-Operation Coverage. ARGUS grounds Stage 1 in CPG invocation nodes. Exact matching covers APIs represented in the knowledge base, and the LLM fallback can recover project-specific calls, but the current design is still less suitable for vulnerabilities whose critical operation is expressed primarily through language primitives rather than an invocation.

Program-Analysis Coverage. The context produced by Stage 2 depends on the precision of the underlying CPG and on the configured analysis bounds. Indirect calls, pointer aliases, macros, project-specific semantics, and dependencies beyond the bounded interprocedural scope may be only partially recovered. As a result, a relevant guard, definition, or lifetime relation can remain outside the context supplied to Stage 3.

LLM-Based Validation. The Prosecutor, Defender, and Adjudicator remain LLM-based components. Restricting them to a shared program context reduces unsupported reasoning but does not provide formal guarantees. A Benign verdict should therefore be interpreted as the absence of an established vulnerability under the recovered evidence, not as a proof of program safety.

5.2 Threats to Validity

Internal Validity. The reported results depend on implementation choices in both ARGUS and the reproduced baseline, including the Joern version, graph-extraction configuration, context budget, prompt templates, model checkpoint,

and decoding parameters. We reduce this threat by freezing the evaluation sample identities before inference, using the same ARGUS configuration across samples, and recording the exact baseline snapshot and inference configuration used in the reproduction. Ground-truth labels, patch differences, vulnerable–fixed relations, and CVE descriptions are not exposed to ARGUS during inference or candidate selection.

Data Selection and Leakage. The released PrimeVul and SVEN artifacts used in our experiments do not fully match the aggregate dataset composition reported in the original LLMxCPG paper. We therefore report the released-artifact distributions explicitly and use the same frozen artifacts for all reproduced comparisons rather than modifying them to match published counts. Exact and normalized-code duplicates are audited before evaluation, and vulnerable–fixed relationships are retained only as evaluation metadata. The relatively small PrimeVul artifact also limits the stability of estimates such as Recall; consequently, the reported results characterize the evaluated artifact rather than the full PrimeVul distribution.

Knowledge-Base Bias. ARGUS relies on a manually curated security knowledge base for deterministic candidate identification. If this knowledge were constructed or revised from the final test samples, the resulting performance could overestimate generalization. To limit this risk, the knowledge base and its API–CWE mappings are frozen before the final evaluation, and no entry is changed in response to the reported test predictions. We separately distinguish exact knowledge-base matches from the LLM fallback so that dependence on curated knowledge can be measured in future coverage and ablation analyses.

Construct Validity. Accuracy, Precision, Recall, and F1-score measure function-level classification but do not establish that ARGUS identifies the exact vulnerability location or reconstructs the complete triggering condition. A correct binary prediction can coexist with incomplete intermediate reasoning. The pipeline therefore retains candidate locations, reconstructed contexts, and agent outputs for stage-level inspection, but a dedicated localization benchmark would be required to evaluate reconstruction quality directly.

Conclusion Validity. Pipeline failures and unequal CWE representation can distort aggregate metrics if their denominators are not stated explicitly. We therefore report the number of valid predictions used for each metric and distinguish reproduced results from values reported in prior publications. Broader evaluation, repeated sampling, additional LLM backbones, and a completed same-sample PrimeVul reproduction of LLMxCPG are required before drawing stronger conclusions about generalization beyond the released artifacts used here.

6 Conclusion

We presented ARGUS, a program-grounded framework that separates sensitive operation identification, program context construction, and vulnerability val-

idation. ARGUS grounds candidate discovery in CPG call sites and a frozen security knowledge base, reconstructs candidate-specific data, control, source-level, and bounded interprocedural context, and asks opposing LLM roles to evaluate supporting and counter-evidence over the same program evidence. Experiments on the released PrimeVul and SVEN artifacts show a recall-oriented prediction profile, while the explicit provenance analysis also reveals that the released LLMxCPG datasets do not fully match the aggregate distributions reported in the original paper. We therefore distinguish reported baselines from locally reproduced comparisons and treat false-positive control, broader context recovery, knowledge-base coverage, and stronger same-sample reproduction as the main directions for further evaluation.

References

1. Ding, Y., Fu, Y., Ibrahim, O., Sitawarin, C., Chen, X., Alomair, B., Wagner, D., Ray, B., Chen, Y.: Vulnerability detection with code language models: How far are we? In: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). pp. 1729–1741. IEEE, Ottawa, ON, Canada (2025). https://doi.org/10.1109/ICSE55347.2025.00038, https://arxiv.org/abs/2403.18624
2. Du, X., Zheng, G., Wang, K., Zou, Y., Wang, Y., Deng, W., Feng, J., Liu, M., Chen, B., Peng, X., Ma, T., Lou, Y.: Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG. ACM Transactions on Software Engineering and Methodology **35**(10), 1–25 (2026). https://doi.org/10.1145/3797277, https://doi.org/10.1145/3797277
3. Guo, J., Wang, C., Xu, X., Su, Z., Zhang, X.: RepoAudit: An autonomous LLM-agent for repository-level code auditing. In: Proceedings of the 42nd International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 267, pp. 21083–21100. PMLR (2025), https://proceedings.mlr.press/v267/guo25n.html
4. He, J., Vechev, M.: Large language models for code: Security hardening and adversarial testing. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 1865–1879. ACM (2023). https://doi.org/10.1145/3576915.3623175, https://arxiv.org/abs/2302.05319
5. Lekssays, A., Mouhcine, H., Tran, K., Yu, T., Khalil, I.: LLMxCPG: Context-Aware vulnerability detection through code property Graph-Guided large language models. In: 34th USENIX Security Symposium (USENIX Security 25). pp. 489–507. USENIX Association, Seattle, WA (Aug 2025), https://www.usenix.org/conference/usenixsecurity25/presentation/lekssays
6. Li, Y., Joshi, K., Wang, X., Wong, E.: MAVUL: Multi-agent vulnerability detection via contextual reasoning and interactive refinement. In: 2025 IEEE 7th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA). pp. 314–324. IEEE, Pittsburgh, PA, USA (2025). https://doi.org/10.1109/TPS-ISA67132.2025.00040, https://arxiv.org/abs/2510.00317
7. Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y.: VulDeePecker: A deep learning-based system for vulnerability detection. In: Network and Distributed System Security Symposium (NDSS). Internet Society (2018). https://doi.org/10.14722/ndss.2018.23158, https://doi.org/10.14722/ndss.2018.23158

8. Li, Z., Dutta, S., Naik, M.: Iris: Llm-assisted static analysis for detecting security vulnerabilities (2025), https://arxiv.org/abs/2405.17238
9. Steenhoek, B., Rahman, M.M., Jiles, R., Le, W.: An empirical study of deep learning models for vulnerability detection. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). pp. 2237–2248. IEEE (2023). https://doi.org/10.1109/ICSE48619.2023.00188, https://arxiv.org/abs/2212.08109
10. Ullah, S., Han, M., Pujar, S., Pearce, H., Coskun, A., Stringhini, G.: LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks. In: 2024 IEEE Symposium on Security and Privacy (SP). pp. 862–880. IEEE (2024). https://doi.org/10.1109/SP54263.2024.00210, https://arxiv.org/abs/2312.12575
11. Wang, Z., Li, G., Li, J., Zhu, H., Jin, Z.: VulAgent: Hypothesis-validation driven multi-agent architecture for vulnerability detection. In: Liakata, M., Moreira, V.P., Zhang, J., Jurgens, D. (eds.) Findings of the Association for Computational Linguistics: ACL 2026. pp. 18598–18616. Association for Computational Linguistics, San Diego, California, United States (Jul 2026). https://doi.org/10.18653/v1/2026.findings-acl.928, https://aclanthology.org/2026.findings-acl.928/
12. Widyasari, R., Weyssow, M., Irsan, I.C., Ang, H.W., Liauw, F., Ouh, E.L., Shar, L.K., Kang, H.J., Lo, D.: Let the trial begin: A mock-court approach to vulnerability detection using LLM-based agents. In: Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering. pp. 1030–1042. ICSE '26, ACM (Apr 2026). https://doi.org/10.1145/3744916.3773256, https://doi.org/10.1145/3744916.3773256
13. Wu, Z., Xu, J., Peng, Y., Chong, C.Y., Jia, X.: MulVul: Retrieval-augmented multi-agent code vulnerability detection via cross-model prompt evolution. In: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 8660–8673. Association for Computational Linguistics, San Diego, California, United States (2026). https://doi.org/10.18653/v1/2026.acl-long.391, https://doi.org/10.18653/v1/2026.acl-long.391
14. Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y.: Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: Advances in Neural Information Processing Systems. vol. 32. Curran Associates, Inc. (2019), https://proceedings.neurips.cc/paper_files/paper/2019/file/49265d2447bc3bbfe9e76306ce40a31f-Paper.pdf